Finding the Sweet Spot: Speed vs. Fidelity in Encrypted LLMs via Proxy Simulation

Rajan Savani[1], Anwar Benhnini[1], André Bauer[2]

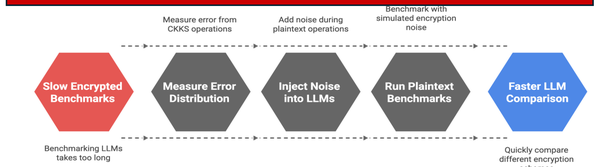
[1]Illinois Institute of Technology [2]Illinois Institute of Technology (Mentor)

ILLINOIS TECH
College of Computing

Background & Motivation

- Large language models (LLMs) are too resource-intensive for most users to run locally.
- Decentralized LLMs address this by distributing inference across peers, improving accessibility while reducing hardware demands
- However, this raises privacy concerns, since user prompts are sent to unknown nodes.
- Fully homomorphic encryption (FHE) allows for computation on encrypted data, protecting privacy without revealing inputs.
- But FHE is slow, introduces noise, and requires approximating nonlinear activations (e.g., GeLU) with polynomials, which can reduce model accuracy.
- We introduce a ~100x faster proxy for encrypted inference that simulates CKKS noise and guides design tradeoffs in polynomial-approximated LLMs.

Method Overview



- Record CKKS noise by encrypting activations from a forward pass and logging the residual error.
- Inject sampled noise into a standard GPT-2 during inference using runtime hooks.
- Evaluate fidelity using lm-eval-harness with noisy models.

Polynomial Approximation

- Nonlinear activations like GeLU must be replaced by polynomial fits under CKKS encryption.
- Piecewise polynomial approximations (Dong et al., 2023) achieve good fidelity overall, but error can spike in certain intervals.

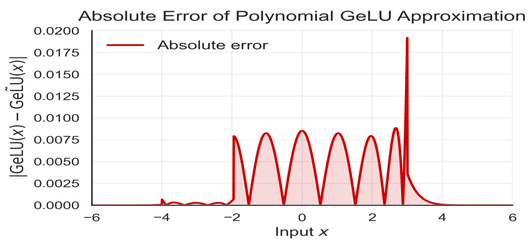


Figure 1. Absolute error between GeLU and Dong et al.'s approximation on [-6, 6]. Error is near zero outside [-4, 3], but volatile in the main interval where degree-3 and degree-6 pieces are active.

- To capture realistic behavior, we logged ~10 5 GeLU inputs from GPT-2 during forward passes.
- For each input, we computed two errors separately:
  - Polynomial approx. error: $\varepsilon_{poly} = P_d(x) - f(x)$
  - CKKS encryption error: $\varepsilon_{ckks} = \tilde{P}_d(x) - P_d(x)$
- These residuals were stored as noise pools and injected into GPT-2 during plaintext inference, simulating encrypted behavior.
- This allows benchmarks (ARC-Easy, PIQA, HellaSwag, etc.) to run ~100x faster while still reflecting realistic CKKS fidelity loss.

Results

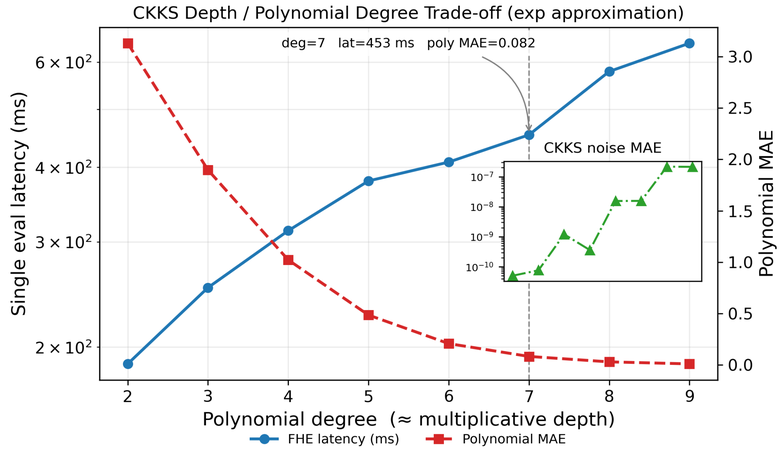


Figure 2. Tradeoff between polynomial degree, approximation error, and evaluation latency for Softmax (exp(x), Chebyshev fits on [-4, 4]). Approximation error (red) decreases with degree but quickly plateaus, while latency (blue) grows linearly. CKKS noise (green) remains negligible across all degrees.

- Approximated exp(x) on [-4, 4] with Chebyshev polynomials of degree 2-9 to replace Softmax under CKKS encryption.
- Approximation error shrinks as degree increases, but with diminishing returns.
- Latency of evaluating the polynomial grows roughly linearly with degree.
- CKKS encryption noise grows with multiplicative depth but is magnitudes lower than the error generated by polynomial approximations.

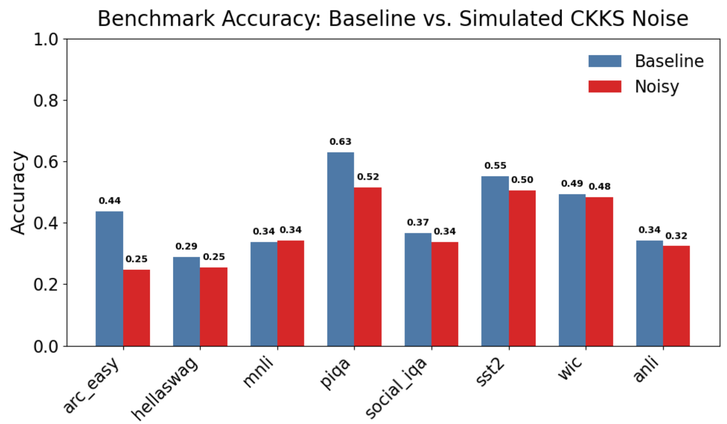


Figure 3. Accuracy on 8 NLP benchmarks comparing plaintext GPT-2 (blue) and proxy-simulated noisey model with degree-7 Softmax polynomial (red). Proxy reveals fidelity drop without full CKKS evaluation.

- Proxy evaluated GPT-2 on 8 standard NLP tasks using lm-eval-harness
- Degree-7 Chebyshev polynomial used for Softmax approximation.
- Average accuracy drop of ~5.4%; largest loss on ARC-Easy (~19%), no significant difference in performance for some tasks (e.g., MNLI).
- Demonstrates how encrypted-LLM developers can rapidly test fidelity impacts of polynomial choices on benchmarks they care about.

Discussion

- The proxy shows that encrypted LLM tradeoffs can be explored in hours, not weeks, by adding realistic noise to plaintext models.
- Different applications may prioritize different points on the tradeoff curve:
  - Latency-critical services (e.g., chatbots) can stop at lower-degree polynomials for faster response.
  - Precision-focused settings (e.g., medical or legal analysis) may accept deeper circuits for marginal accuracy gains.
- Limitations:
  - Proxy models per-layer noise only (excludes bootstrapping and I/O costs that can affect performance significantly).
  - Residuals collected from random token inputs; task-specific distributions may produce different noise patterns.
  - Latency estimates extrapolated from pilot CKKS runs → full benchmark still needed for comparison

Conclusion

- Introduced a noise-injection proxy that predicts encrypted LLM fidelity losses ~100x faster than full CKKS inference.
- Enables developers to map accuracy-latency tradeoffs quickly and identify the practical operating points for their applications.
- Workflow is reusable and sharable, lowering the barrier for encrypted inference research across different models and schemas.
- Future work: extend the proxy to capture bootstrapping overhead and validate predictions against models encrypted end-to-end with CKKS.

Acknowledgments

- This work is supported in part by the National Science Foundation OAC-2150500 award.

References

[1] A. Borzunov, D. Baranchuk, T. Dettmers, M. Ryabinin, Y. Belkada, A. Chumachenko, P. Samygin, and C. Raffel, *Petals: Collaborative inference and fine-tuning of large models*, arXiv preprint arXiv:2209.01188, 2023.

[2] L. de Castro, D. Escudero, A. Agrawal, A. Polychroniadou, and M. Veloso, *EncryptedLLM: Privacy-preserving large language model inference via GPU-accelerated fully homomorphic encryption*, in *Forty-second International Conference on Machine Learning*, 2025.

[3] J. H. Cheon, A. Kim, M. Kim, and Y. Song, *Homomorphic Encryption for Arithmetic of Approximate Numbers*, in *ASIACRYPT*, pp. 409–437, 2017.

[4] EleutherAI, *lm-eval-harness*, https://github.com/EleutherAI/lm-evaluation-harness.

[5] HuggingFace Datasets: ARC-Easy, HellaSwag, PIQA, Social IQa, MNLI, SST-2, ANLI, WiC.

Scan for poster and paper!